

**Menu-Driven Program to Verify the Associative Property of
Matrix Addition and Matrix Multiplication using C**

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Aim

To write a menu-driven C program, implemented using separate reusable functions, that reads variable-length two-dimensional integer matrices and verifies:

- The Associative Property of Matrix Addition: $(A + B) + C = A + (B + C)$
- The Associative Property of Matrix Multiplication: $(A \times B) \times C = A \times (B \times C)$

2. Objective

- To understand dynamic memory allocation for variable-sized 2-D arrays in C.
- To practice modular programming by dividing the task into small, single-purpose functions.
- To implement and verify a fundamental mathematical property (associativity) using matrix operations.
- To design and use a menu-driven interface for a multi-option console program.

3. Problem Statement

Design a menu-driven C program with the following options:

1. Read three variable-length 2-D integer matrices A, B and C, each of dimension $(m \times n)$, and verify the associative property of matrix addition: $(A + B) + C = A + (B + C)$.
2. Read three variable-length 2-D integer matrices with dimensions A $(m \times n)$, B $(n \times p)$ and C $(p \times q)$, and verify the associative property of matrix multiplication: $(A \times B) \times C = A \times (B \times C)$.
3. Exit the program.

For each operation, the matrices obtained on both sides of the equation must be displayed and the program must state whether the associative property is verified.

4. Algorithm

4.1 Main Menu

1. Start.
2. Display the menu with options 1 (Addition), 2 (Multiplication) and 3 (Exit).
3. Read the user's choice.
4. If choice = 1, call `verifyAdditionAssociativity()`.
5. If choice = 2, call `verifyMultiplicationAssociativity()`.
6. If choice = 3, terminate the program.
7. Repeat steps 2–6 until the user selects Exit.

4.2 Verify Associative Property of Addition

1. Read the order $(m \times n)$ of the matrices.
2. Dynamically allocate and read matrices A, B and C of size $(m \times n)$.
3. Compute $(A + B)$, then add C to obtain LHS = $(A + B) + C$.
4. Compute $(B + C)$, then add A to obtain RHS = $A + (B + C)$.
5. Display matrices A, B, C, and both sides of the equation.

6. Compare LHS and RHS element-wise. If all elements match, print “Associative property VERIFIED”, otherwise print “NOT verified”.
7. Free the dynamically allocated memory.

4.3 Verify Associative Property of Multiplication

1. Read dimensions m, n, p, q for $A (m \times n)$, $B (n \times p)$ and $C (p \times q)$.
2. Dynamically allocate and read matrices A, B and C .
3. Compute $(A \times B)$, then multiply the result by C to obtain $LHS = (A \times B) \times C$.
4. Compute $(B \times C)$, then multiply A by the result to obtain $RHS = A \times (B \times C)$.
5. Display matrices A, B, C , and both sides of the equation.
6. Compare LHS and RHS element-wise and print whether the property is verified.
7. Free the dynamically allocated memory.

5. Functions Used

- `createMatrix()` – dynamically allocates memory for an $m \times n$ matrix.
- `freeMatrix()` – releases dynamically allocated matrix memory.
- `readMatrix()` – reads matrix elements from the user with input validation.
- `printMatrix()` – displays a matrix in a formatted layout.
- `addMatrices()` – returns the sum of two matrices.
- `multiplyMatrices()` – returns the product of two matrices.
- `compareMatrices()` – checks whether two matrices are identical.
- `verifyAdditionAssociativity()` – drives Option 1 of the menu.
- `verifyMultiplicationAssociativity()` – drives Option 2 of the menu.
- `readPositiveInt()` – validated integer input helper.

6. Source Code

File: *matrix_associative.c*

```

/*
 * =====
 * Program : Menu-Driven Verification of Associative
 *           Property of Matrix Addition and Multiplication
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 * =====
 *
 * Description:
 * This program uses dynamically allocated (variable length)
 * 2-D integer arrays to:
 * 1. Verify  $(A+B)+C = A+(B+C)$  for matrices of size  $m \times n$ 
 * 2. Verify  $(A \times B) \times C = A \times (B \times C)$  for matrices  $A(m \times n)$ ,
 *     $B(n \times p)$ ,  $C(p \times q)$ 
 * 3. Exit the program
 *
 * All operations are implemented as separate, reusable
 * functions (creation, reading, printing, addition,
 * multiplication, comparison and freeing of memory).
 * =====
 */

#include <stdio.h>
#include <stdlib.h>

/* ----- Function Prototypes ----- */
int **createMatrix(int rows, int cols);
void freeMatrix(int **mat, int rows);
void readMatrix(int **mat, int rows, int cols, const char *name);
void printMatrix(int **mat, int rows, int cols, const char *label);
int **addMatrices(int **A, int **B, int rows, int cols);
int **multiplyMatrices(int **A, int **B, int r1, int c1, int c2);
int compareMatrices(int **A, int **B, int rows, int cols);
int readPositiveInt(const char *prompt);

void verifyAdditionAssociativity(void);
void verifyMultiplicationAssociativity(void);

/* =====
 * main() : Displays the menu and calls the appropriate
 *          function based on the user's choice.
 * =====
 */
int main(void)
{
    int choice;

    do
    {
        printf("\n=====\\n");
        printf(" MENU - ASSOCIATIVE PROPERTY OF MATRICES\\n");
        printf("=====\\n");
        printf(" 1. Verify Associative Property of Matrix Addition\\n");
        printf("     $((A + B) + C = A + (B + C))\\n$ ");
        printf(" 2. Verify Associative Property of Matrix Multiplication\\n");
        printf("     $((A \times B) \times C = A \times (B \times C))\\n$ ");
        printf(" 3. Exit\\n");
        printf("=====\\n");
        choice = readPositiveInt("Enter your choice (1-3): ");

        switch (choice)
        {
            case 1:
                verifyAdditionAssociativity();

```

```

        break;
    case 2:
        verifyMultiplicationAssociativity();
        break;
    case 3:
        printf("\nExiting program. Thank you!\n");
        break;
    default:
        printf("\nInvalid choice! Please enter 1, 2 or 3.\n");
    }
} while (choice != 3);

return 0;
}

/* =====
 * readPositiveInt() : Reads and validates a positive integer
 * =====
 */
int readPositiveInt(const char *prompt)
{
    int value;
    int valid;

    do
    {
        printf("%s", prompt);
        valid = scanf("%d", &value);

        if (valid != 1)
        {
            /* Clear invalid input from the buffer */
            while (getchar() != '\n')
                ;
            printf("Invalid input! Please enter an integer.\n");
            value = -1;
        }
        else if (value <= 0)
        {
            printf("Please enter a positive integer.\n");
        }
    } while (value <= 0);

    return value;
}

/* =====
 * createMatrix() : Dynamically allocates memory for a
 *                  'rows x cols' matrix (variable length array)
 * =====
 */
int **createMatrix(int rows, int cols)
{
    int **mat = (int **)malloc(rows * sizeof(int *));

    if (mat == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < rows; i++)
    {
        mat[i] = (int *)malloc(cols * sizeof(int));
        if (mat[i] == NULL)
        {
            printf("Memory allocation failed!\n");
            exit(EXIT_FAILURE);
        }
    }
}

```

```

    }
    return mat;
}

/* =====
 * freeMatrix() : Releases the dynamically allocated memory
 * =====
 */
void freeMatrix(int **mat, int rows)
{
    for (int i = 0; i < rows; i++)
        free(mat[i]);
    free(mat);
}

/* =====
 * readMatrix() : Reads elements of a matrix from the user
 * =====
 */
void readMatrix(int **mat, int rows, int cols, const char *name)
{
    printf("\nEnter elements of matrix %s (%d x %d), row-wise:\n", name, rows, cols);
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("%s[%d][%d] = ", name, i, j);
            while (scanf("%d", &mat[i][j]) != 1)
            {
                while (getchar() != '\n')
                    ;
                printf("Invalid input! %s[%d][%d] = ", name, i, j);
            }
        }
    }
}

/* =====
 * printMatrix() : Displays a matrix with a given label
 * =====
 */
void printMatrix(int **mat, int rows, int cols, const char *label)
{
    printf("\n%s (%d x %d):\n", label, rows, cols);
    for (int i = 0; i < rows; i++)
    {
        printf("\t");
        for (int j = 0; j < cols; j++)
            printf("%6d ", mat[i][j]);
        printf("\n");
    }
}

/* =====
 * addMatrices() : Returns a newly allocated matrix = A + B
 * =====
 */
int **addMatrices(int **A, int **B, int rows, int cols)
{
    int **sum = createMatrix(rows, cols);

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            sum[i][j] = A[i][j] + B[i][j];

    return sum;
}

/* =====

```

```

* multiplyMatrices() : Returns a newly allocated matrix = A x B
*                      A is (r1 x c1), B is (c1 x c2)
* =====
*/
int **multiplyMatrices(int **A, int **B, int r1, int c1, int c2)
{
    int **product = createMatrix(r1, c2);

    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {
            product[i][j] = 0;
            for (int k = 0; k < c1; k++)
                product[i][j] += A[i][k] * B[k][j];
        }
    }
    return product;
}

/* =====
* compareMatrices() : Returns 1 if both matrices are equal,
*                      else returns 0
* =====
*/
int compareMatrices(int **A, int **B, int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (A[i][j] != B[i][j])
                return 0;
    return 1;
}

/* =====
* verifyAdditionAssociativity() :
* Reads matrices A, B, C of size (m x n) and verifies
* (A+B)+C = A+(B+C)
* =====
*/
void verifyAdditionAssociativity(void)
{
    int m, n;

    printf("\n--- Verification of Associative Property of Matrix Addition ---\n");
    m = readPositiveInt("Enter number of rows (m): ");
    n = readPositiveInt("Enter number of columns (n): ");

    int **A = createMatrix(m, n);
    int **B = createMatrix(m, n);
    int **C = createMatrix(m, n);

    readMatrix(A, m, n, "A");
    readMatrix(B, m, n, "B");
    readMatrix(C, m, n, "C");

    printMatrix(A, m, n, "Matrix A");
    printMatrix(B, m, n, "Matrix B");
    printMatrix(C, m, n, "Matrix C");

    /* LHS = (A + B) + C */
    int **AplusB = addMatrices(A, B, m, n);
    int **LHS = addMatrices(AplusB, C, m, n);

    /* RHS = A + (B + C) */
    int **BplusC = addMatrices(B, C, m, n);
    int **RHS = addMatrices(A, BplusC, m, n);

```

```

printf("\n----- LHS : (A + B) + C ----- \n");
printMatrix(AplusB, m, n, "(A + B)");
printMatrix(LHS, m, n, "(A + B) + C");

printf("\n----- RHS : A + (B + C) ----- \n");
printMatrix(BplusC, m, n, "(B + C)");
printMatrix(RHS, m, n, "A + (B + C)");

if (compareMatrices(LHS, RHS, m, n))
    printf("\nResult: (A + B) + C = A + (B + C)\n"
           "Associative property of matrix ADDITION is VERIFIED.\n");
else
    printf("\nResult: (A + B) + C != A + (B + C)\n"
           "Associative property of matrix ADDITION is NOT verified.\n");

/* Free all dynamically allocated memory */
freeMatrix(A, m);
freeMatrix(B, m);
freeMatrix(C, m);
freeMatrix(AplusB, m);
freeMatrix(LHS, m);
freeMatrix(BplusC, m);
freeMatrix(RHS, m);
}

/* =====
 * verifyMultiplicationAssociativity() :
 * Reads matrices A(m x n), B(n x p), C(p x q) and verifies
 * (A x B) x C = A x (B x C)
 * =====
 */
void verifyMultiplicationAssociativity(void)
{
    int m, n, p, q;

    printf("\n--- Verification of Associative Property of Matrix Multiplication --- \n");
    printf("Matrix A -> (m x n), Matrix B -> (n x p), Matrix C -> (p x q)\n");
    m = readPositiveInt("Enter m (rows of A): ");
    n = readPositiveInt("Enter n (cols of A = rows of B): ");
    p = readPositiveInt("Enter p (cols of B = rows of C): ");
    q = readPositiveInt("Enter q (cols of C): ");

    int **A = createMatrix(m, n);
    int **B = createMatrix(n, p);
    int **C = createMatrix(p, q);

    readMatrix(A, m, n, "A");
    readMatrix(B, n, p, "B");
    readMatrix(C, p, q, "C");

    printMatrix(A, m, n, "Matrix A");
    printMatrix(B, n, p, "Matrix B");
    printMatrix(C, p, q, "Matrix C");

    /* LHS = (A x B) x C --> (m x p) x (p x q) = (m x q) */
    int **AB = multiplyMatrices(A, B, m, n, p);
    int **LHS = multiplyMatrices(AB, C, m, p, q);

    /* RHS = A x (B x C) --> (m x n) x (n x q) = (m x q) */
    int **BC = multiplyMatrices(B, C, n, p, q);
    int **RHS = multiplyMatrices(A, BC, m, n, q);

    printf("\n----- LHS : (A x B) x C ----- \n");
    printMatrix(AB, m, p, "(A x B)");
    printMatrix(LHS, m, q, "(A x B) x C");

    printf("\n----- RHS : A x (B x C) ----- \n");

```

```
printMatrix(BC, n, q, "(B x C)");
printMatrix(RHS, m, q, "A x (B x C)");

if (compareMatrices(LHS, RHS, m, q))
    printf("\nResult: (A x B) x C = A x (B x C)\n"
           "Associative property of matrix MULTIPLICATION is VERIFIED.\n");
else
    printf("\nResult: (A x B) x C != A x (B x C)\n"
           "Associative property of matrix MULTIPLICATION is NOT verified.\n");

/* Free all dynamically allocated memory */
freeMatrix(A, m);
freeMatrix(B, n);
freeMatrix(C, p);
freeMatrix(AB, m);
freeMatrix(LHS, m);
freeMatrix(BC, n);
freeMatrix(RHS, m);
}
```

7. Sample Input / Output

7.1 Option 1 – Associative Property of Addition (3×3 matrices)

```
=====
MENU - ASSOCIATIVE PROPERTY OF MATRICES
=====
1. Verify Associative Property of Matrix Addition
   ((A + B) + C = A + (B + C))
2. Verify Associative Property of Matrix Multiplication
   ((A x B) x C = A x (B x C))
3. Exit
=====
Enter your choice (1-3):
--- Verification of Associative Property of Matrix Addition ---
Enter number of rows (m): Enter number of columns (n):
Enter elements of matrix A (3 x 3), row-wise:
A[0][0] = A[0][1] = A[0][2] = A[1][0] = A[1][1] = A[1][2] = A[2][0] = A[2][1] = A[2][2] =
Enter elements of matrix B (3 x 3), row-wise:
B[0][0] = B[0][1] = B[0][2] = B[1][0] = B[1][1] = B[1][2] = B[2][0] = B[2][1] = B[2][2] =
Enter elements of matrix C (3 x 3), row-wise:
C[0][0] = C[0][1] = C[0][2] = C[1][0] = C[1][1] = C[1][2] = C[2][0] = C[2][1] = C[2][2] =
Matrix A (3 x 3):
    1    2    3
    4    5    6
    7    8    9

Matrix B (3 x 3):
    9    8    7
    6    5    4
    3    2    1

Matrix C (3 x 3):
    1    1    1
    1    1    1
    1    1    1

----- LHS : (A + B) + C -----

(A + B) (3 x 3):
    10   10   10
    10   10   10
    10   10   10

(A + B) + C (3 x 3):
    11   11   11
    11   11   11
    11   11   11

----- RHS : A + (B + C) -----

(B + C) (3 x 3):
    10    9    8
     7    6    5
     4    3    2

A + (B + C) (3 x 3):
    11   11   11
    11   11   11
    11   11   11
```

Result: $(A + B) + C = A + (B + C)$
 Associative property of matrix ADDITION is VERIFIED.

```
=====
MENU - ASSOCIATIVE PROPERTY OF MATRICES
=====
1. Verify Associative Property of Matrix Addition
   ((A + B) + C = A + (B + C))
2. Verify Associative Property of Matrix Multiplication
   ((A x B) x C = A x (B x C))
3. Exit
=====
Enter your choice (1-3):
Exiting program. Thank you!
```

7.2 Option 2 – Associative Property of Multiplication (A: 2x3, B: 3x2, C: 2x2)

```
=====
MENU - ASSOCIATIVE PROPERTY OF MATRICES
=====
1. Verify Associative Property of Matrix Addition
   ((A + B) + C = A + (B + C))
2. Verify Associative Property of Matrix Multiplication
   ((A x B) x C = A x (B x C))
3. Exit
=====
Enter your choice (1-3):
--- Verification of Associative Property of Matrix Multiplication ---
Matrix A -> (m x n), Matrix B -> (n x p), Matrix C -> (p x q)
Enter m (rows of A): Enter n (cols of A = rows of B): Enter p (cols of B = rows of C): Enter q (cols
of C):
Enter elements of matrix A (2 x 3), row-wise:
A[0][0] = A[0][1] = A[0][2] = A[1][0] = A[1][1] = A[1][2] =
Enter elements of matrix B (3 x 2), row-wise:
B[0][0] = B[0][1] = B[1][0] = B[1][1] = B[2][0] = B[2][1] =
Enter elements of matrix C (2 x 2), row-wise:
C[0][0] = C[0][1] = C[1][0] = C[1][1] =
Matrix A (2 x 3):
      1      2      3
      4      5      6

Matrix B (3 x 2):
      1      0
      0      1
      1      1

Matrix C (2 x 2):
      2      1
      1      2

----- LHS : (A x B) x C -----

(A x B) (2 x 2):
      4      5
     10     11

(A x B) x C (2 x 2):
     13     14
     31     32
```

----- RHS : A x (B x C) -----

(B x C) (3 x 2):

2	1
1	2
3	3

A x (B x C) (2 x 2):

13	14
31	32

Result: (A x B) x C = A x (B x C)

Associative property of matrix MULTIPLICATION is VERIFIED.

=====

MENU - ASSOCIATIVE PROPERTY OF MATRICES

=====

1. Verify Associative Property of Matrix Addition
((A + B) + C = A + (B + C))
2. Verify Associative Property of Matrix Multiplication
((A x B) x C = A x (B x C))
3. Exit

=====

Enter your choice (1-3):

Exiting program. Thank you!

8. Result / Conclusion

The menu-driven C program was successfully designed, implemented and executed using separate functions for matrix creation, reading, printing, addition, multiplication, and comparison.

Test Case 1 (Addition): For matrices A, B, C of order 3×3 , it was verified that $(A + B) + C = A + (B + C)$, confirming that matrix addition is associative.

Test Case 2 (Multiplication): For matrices A (2×3), B (3×2), C (2×2), it was verified that $(A \times B) \times C = A \times (B \times C)$, confirming that matrix multiplication is associative.

The program correctly handles variable-length (dynamically sized) matrices using pointer-based dynamic memory allocation, and all allocated memory is freed after use to avoid memory leaks.

Hence, the associative properties of matrix addition and matrix multiplication were successfully verified programmatically.